Proof of concept for integrating TORCS within a DDS environment

Apollonio Marco, Bossi Giacomo

August 28, 2025

Contents

1 Project data

- Project supervisor(s): Prof. Vittorio Zaccaria
- Describe in this table the group that is delivering this project:

Last and first name	Person code	Email address
Marco Apollonio	10764083	marco2.apollonio@mail.polimi.it
Giacomo Bossi	10766073	giacomo3.bossi@mail.polimi.it

- Describe here how development tasks have been subdivided among members of the group, e.g.:
  - Bossi configured the Docker environment to simplify the development process.
  - Bossi configured Qemu emulator to run the ECU that runs FreeRTOS
  - Bossi developed the Uart interrupt to handle the reading of the host keyboard
  - Apollonio worked on the integration of the OpenDDS library within the TORCS simulator.
  - Apollonio developed some high level examples to demonstrate the usage of the DDS communication.
  - Apollonio developed the network configuration for the DDS communication on the FreeRTOS ECU.
- Links to the project source code; Put here, if available, links to public repos hosting your project

2 Project description

2 pages max please

2.1 Introduction

The project aims to integrate the TORCS simulator within a DDS environment, in order to test the real time communication between a ECU, being it a real one or a simulated one, and the TORCS simulator.

The project is divided in two Proof of Concepts, each one with a specific goal to achieve:

- The first Proof of Concept aims to send the steering commands from the ECU to the TORCS simulator using The Vehicle Signal Specification (VSS) standard. One modification done to the VSS standard is the usage of the steering wheel as an actuator instead of being a sensor.
- The second Proof of Concept aims to create a torque vectoring algorithm on the ECU so that it receives as an input the yaw rate and the lateral acceleration of the car from the simulator and the angle of the steering wheel from the steering wheel. The ECU will then calculate the torque to be applied to each wheel and send it to the simulator. All this communication will be done using the DDS environment, without the usage of the VSS standard.

2.2 Design and implementation

We managed to implement the First Proof of Concept, with the communication between the ECU and the TORCS simulator working as expected.

For the Second Proof of Concept we managed to link the physical engine of the TORCS simulator to where the pilot need to be implemented, but after analyzing the engine code we realized that the to implement the torque vectoring algorithm we need to modify the engine code to calculate all the velocity and acceleration values based on the torque, because as it is now the torque are just for reading purposes.

2.2.1 Design of First Proof of Concept

For the first Proof Of Concept we decided to create one topics in the DDS environment, using the VSS standard for the steering commands:

- SteeringInformation: it's used to send the steering commands to the TORCS simulator.

The ECU will have a Publisher for the SteeringInformation topic. The TORCS simulator will have a Subscriber for the SteeringInformation.

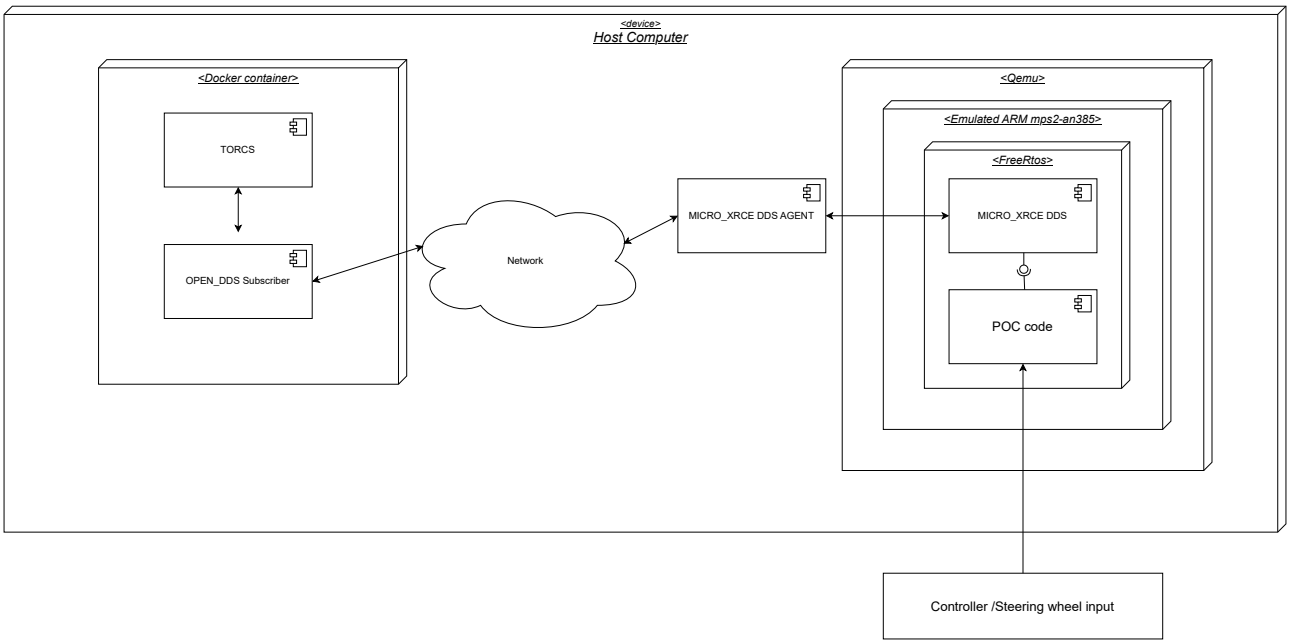


Figure 1: Schematization of the first proof of concept

3 Project outcomes

3.1 Concrete outcomes

3.1.1 Subscriber Implementation

The implementation of the OpenDDS Subscriber is done in a bot called AOS_dds_hwloop. We implemented specific classes that are used in the bot methods when the race is being created and when the race is started.

When TORCS invokes the method `AOS_dds_hwloop::newrace`, the bot will initialize a static Subscriber object that will do all the configuration needed to create the DDS Domain and Topic to which then it will subscribe and listen to the messages sent by the ECU.

Then when TORCS invokes the method `AOS_dds_hwloop::drive`, the bot will call the method `Subscriber::pop_command` that return the value in the queue, if no message is present then the steering value is the last available one that was received, based on the fact that a steering wheel if left without user input will return to it's base position that is 0 degree.

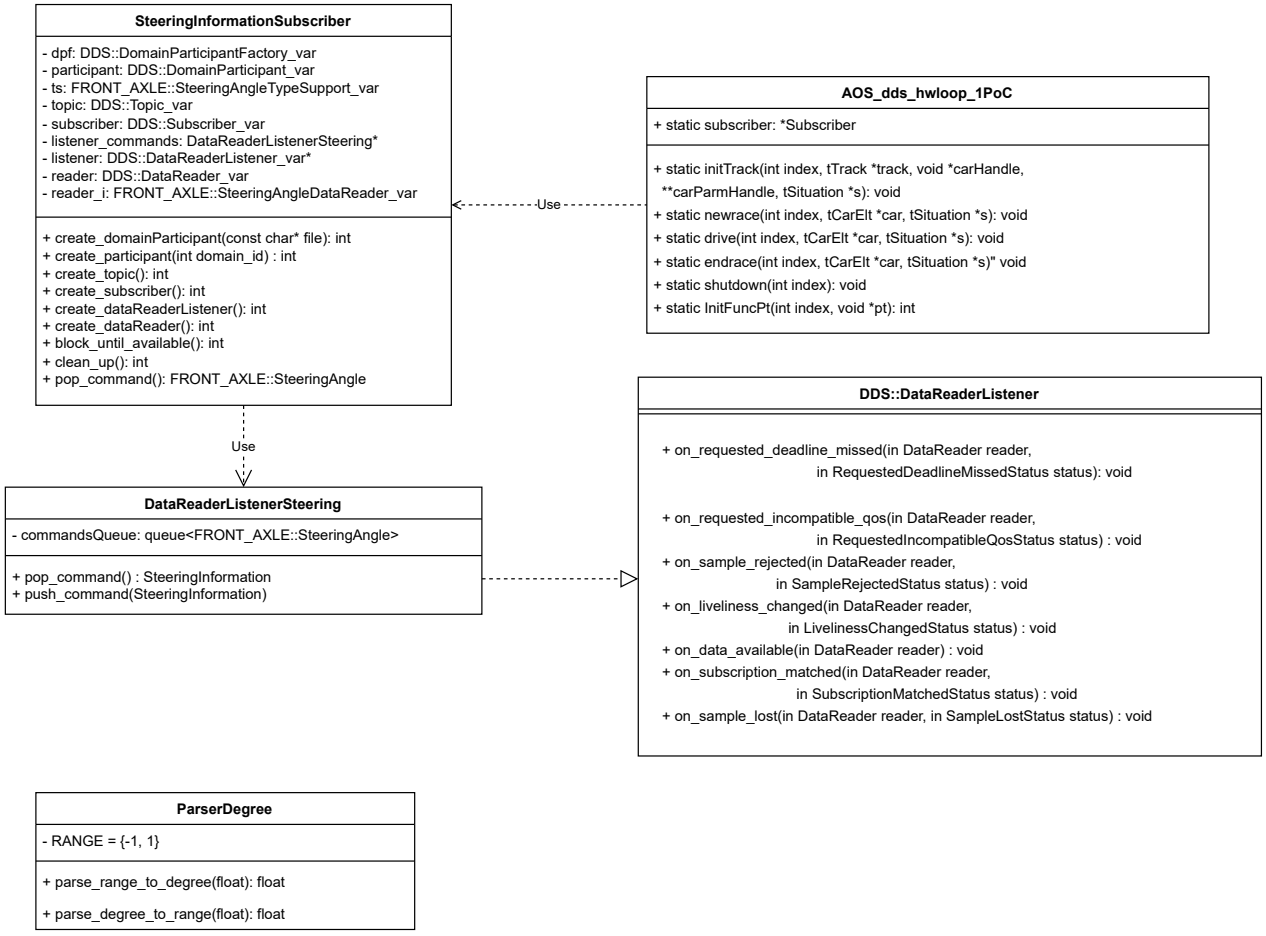


Figure 2: UML diagram of the Subscriber implementation

The topic in the TORCS code has been defined in the file SteeringInformation.idl then to generate the source code from it we created an MPC file that then generated a specific Makefile to create all this files:

- SteeringInformationC.cpp
- SteeringInformationC.h
- SteeringInformationC.inl
- SteeringInformationS.cpp
- SteeringInformationS.h
- SteeringInformationTypeSupport.idl
- SteeringInformationTypeSupportC.cpp
- SteeringInformationTypeSupportC.h
- SteeringInformationTypeSupportC.inl

- SteeringInformationTypeSupportImpl.cpp
- SteeringInformationTypeSupportImpl.h
- SteeringInformationTypeSupportS.cpp
- SteeringInformationTypeSupportS.h

3.1.2 Publisher Implementation and ECU code design

The MCU code has has two functional requirements to fulfill:

- Being able to receive an input from the user and process it to create the appropriate values to add onto the DDS topic.
- Using the DDS library to publish the steering angle data for the TORCS simulator.

In order to receive data we decided to use the UART interface and read from a stream of chars. This is because QEMU offers different ways to expose such interface , like stdin or a tcp port, which makes using the code more flexible. In order to read the stream of chars we decided, given the real-time nature of the application, that a polling driver was a bad idea due to the waste of clock cycles. Avoiding wasted clock cycles is a principle that guided the entire application design, not just the UART interface. As a consequence we decided to use an interrupt-driven approach to handle the incoming data.

The interrupt priority was set to 5, one above the System Call interrupt priority, which is the highest priority that ensures the proper functioning of FreeRTOS ISR safe functions. The interrupt is very simple it just pushes the chars onto a queue so that another task will process them.

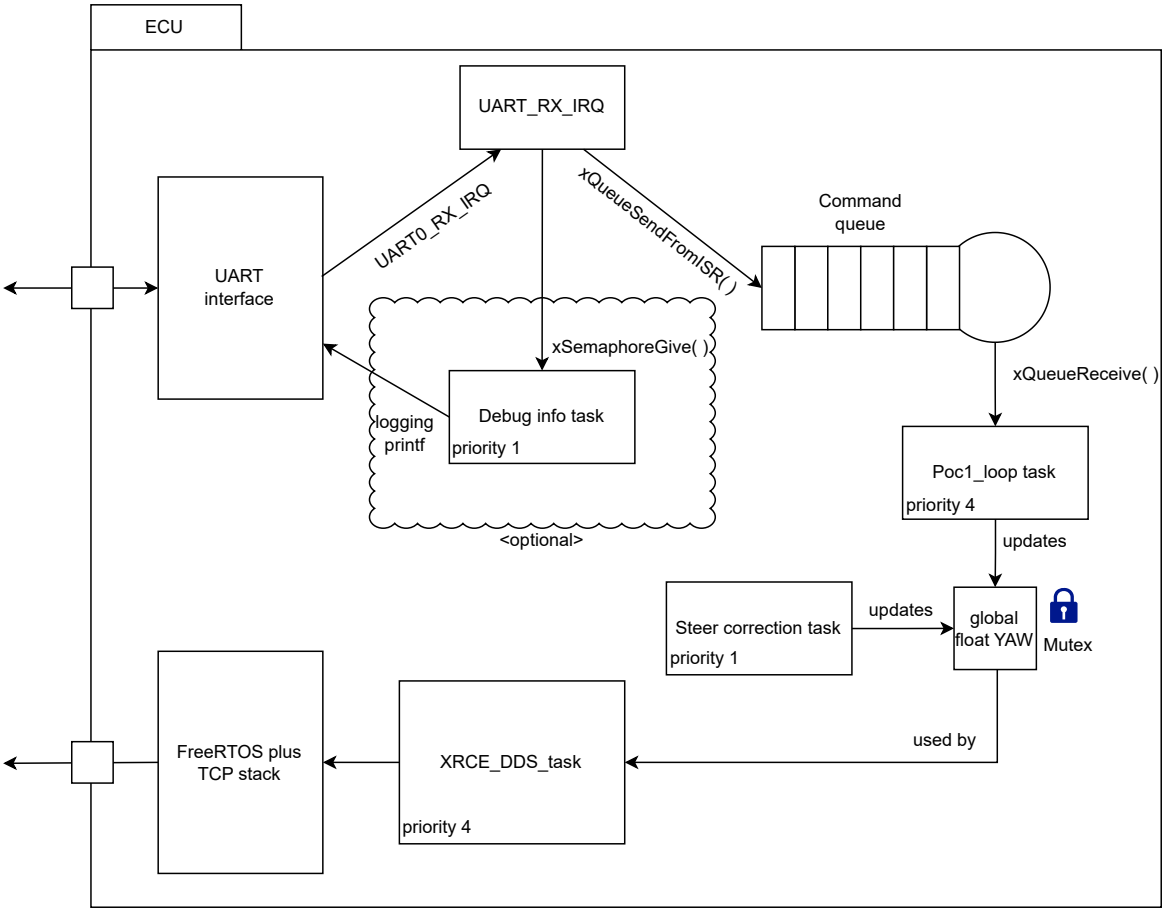
About the DDS communication, we decided to use the XRCE-DDS library to handle the communication with the TORCS simulator due to many different reasons:

- The XRCE-DDS library is lightweight and designed for resource-constrained environments, making it suitable for our MCU due to a lower resource usage compared to other DDS implementations.
- The XRCE implementation has direct support to FreeRTOS, which simplifies enormously the integration because other implementations rely on a full POSIX API which FreeRTOS does not have.

To enable DDS we first needed a working TCP/IP stack, so it was necessary to include into the project the library and code of the FreeRTOS Plus TCP along side the XRCE-DDS library. Into the code it was necessary to enable the Ethernet IRQ and set it's priority. This priority is also 5 for the same reason as the UART interrupt. Another thing that needed to be done was the dimension of the stack size of the task that handles the network, it was set as 8 times the minimum stack size. The minimum stack size was set in the FreeRTOSConfigFile as 512. The last thing needed to enable communication was to configure QEMU to do port forwarding between the virtual board and the host machine. As briefly mentioned before the communication should use the VSS standard. So a .idl file obtained from the standard is used to construct the type for the topic.

3.2 Tasks and Synchronization

Here is a schematization of the tasks and how they are synchronized:


ECU
UART_RX_IRQ
UART0_RX_IRQ
xQueueSendFromISR( )
Command
queue
UART
interface
xSemaphoreGive( )
xQueueReceive( )
logging
printf
Debug info task
priority 1
Poc1_loop task
priority 4
<optional>
updates
Steer correction task
priority 1
updates
global
float YAW
Mutex
used by
FreeRTOS plus
TCP stack
XRCE_DDS_task
priority 4

The picture shows that there are two high priority tasks, the POC1_loop that updates the YAW variable according to the received data and the DDS_task that is responsible for publishing the data to the DDS topic. The POC1_loop task is triggered by the UART interrupt as waiting on the queue is blocking until new data arrives. YAW updates are controlled with a mutex to ensure no race conditions.

The DDS_task is designed to supply a constant stream of messages and so it tries to acquire the mutex for a maximum of 100ms before defaulting and sending again the last value. This ensures a smooth flow of data without gaps under all circumstances.

Two low priority tasks are also present, as extra "accessories" to the application. These tasks are Steer_correction and Debug_info. Steer_correction is used to slowly turn the YAW angle towards zero, while Debug_info (which can be deactivated by setting `DEBUG_INFO = 0`) is used to display debugging information like received chars and YAW value. Debug_info waits for a semaphore given by the UART interrupt to run, so it will run at most once per interrupt, displaying the updated data.

The fact that the Steer_correction task is low priority might look like it would cause issues if it is preempted by higher priority tasks that needs the mutex while Steer_correction is holding it. However FreeRTOS offers a priority inheritance mechanism that given the shortness of the critical session is a good enough solution.

3.3 Learning outcomes

This project pushed us not just to better understand the workings of FreeRTOS and real-time systems and the structure of cpp projects but also forced us to learn in a more advanced way a lot of tools like Make and GDB that were mandatory for the development process. We already knew about these tools and their purpose but we either never used them directly or used them for trivial stuff. An example of this is the Make rule:

```
print-% :  ; @echo $* = $($*).
```

This rule we learned proved to be extremely useful for troubleshooting with Make variables.

Some more detailed examples are:

- We both had to deal with non-trivial build systems for projects. Marco had to work on how to integrate the OpenDDs library to the TORCS codebase and it required significant effort in adapting the build system. Giacomo had to adapt the FreeRTOS Makefile to work in project folder structure. Identifying all the necessary files from FreeRtos, FreeRtos plus TCP, XRCE_DDS and making sure they were getting linked correctly alongside our files while also keeping an organized structure to the project requires some effort.
- Giacomo had to use GDB to debug an issue during the integration of FreeRTOS plus TCP to the base FreeRTOS codebase. More details in the Problems encountered section. This already proved to be a useful skill as we had to use GDB again for the Computer Security course challenges.
- Giacomo has learned how to interface with devices at different levels of abstraction, register level and OS level. The UART driver in the QEMU application required to use bit manipulating techniques typical of register level drivers programming. Instead when writing the controller_reader program an higher level of interface was used, relying on the OS abstractions provided by Linux.
- Marco has learned how does Makefile work and how different files can be nested to compile a single project, he discovered the configuration options available in Makefile config file. He is now able to create more complex build systems and understand the dependencies between different source files.
- Marco has learned how the DDS protocol works, the possible configuration available and the different compatibilities between the different distribution as OpenDDS and XRCE-DDS.

3.4 Existing knowledge

What courses you have followed (not only AOS) did help you in doing this project and why? Do you have any suggestions on improving the AOS course with topics that would have made it easier for you?

For what concerns me, Giacomo, AOS and its predecessor ACSO are the courses that helped me the most in this project because of obvious reasons. The only other course that I feel like it's worth mentioning is the seemingly disconnected Robotics course. The course of robotics introduced me to Docker, C++, and working in a Linux environment, all of which were very useful in this project. Also the ROS framework used in the Robotics course is based on DDS , so I found some similarities in the way they work.

3.5 Problems encountered

- As mentioned before, Giacomo encountered a problem when integrating the FreeRTOS plus TCP library to the base FreeRTOS codebase. The problem was caused by the fact that the Interrupt Vector Table used in the base FreeRTOS did not contain the Ethernet IRQ handler. As a consequence after calling `FreeRTOS_IPInit_Multi()` to initialize the network stack, the Ethernet IRQs were crashing the application due to a null pointer exception. The error seemed to be triggered by `FreeRTOS_IPInit_Multi()` since it will trigger before it returned. The first thing he looked at is if whether he gathered all of the FreeRTOS plus TCP files and correctly included in the project. After that he looked at possible issues with the drivers of the NIC but found nothing. In order to pinpoint the true issue, Giacomo had to spend quite some time on GDB to debug the problem. The fix was just adding a reference to the Ethernet IRQ handler in the Interrupt Vector Table at the correct position.
- Giacomo encountered another minor starvation issue. Due to bad error handling in our code, to prevent the DDS-task from returning after a setup error (ex. agent was not running) we were wrongly using infinite loops. If this loops were reached they starved completely all of lower priority tasks. This issue was not affecting normal behavior so it wasn't caught early. Now on error the task safely destroys itself by calling `vTaskDelete(NULL)`.
- Marco encountered some issues with the OpenDDS library, specifically with the building process and the configuration options available in the library. because in the documentation it was always done with automatic systems that generated the specific file needed, but due to the already existing Makefile used by TORCS he needed a custom solution to integrate the library properly.
- Marco also encountered another problem while trying to understand the feasibility of the Second Proof of Concept, because he needed to link the physical engine shared object to the pilot, but he was unsure about the correct linking process and the dependencies involved. He discovered the existence of specific linux commands such as `ldd` that can be used to check the dependencies of a shared object and how to link them correctly.

4 Honor Pledge

(**This part cannot be modified and it is mandatory to sign it**)

I/We pledge that this work was fully and wholly completed within the criteria established for academic integrity by Politecnico di Milano (Code of Ethics and Conduct) and represents my/our original production, unless otherwise cited.

I/We also understand that this project, if successfully graded, will fulfill part B requirement of the Advanced Operating System course and that it will be considered valid up until the AOS exam of Sept. 2022.

Group Students' signatures
Apollonio Marco
Bossi Giacomo